

CSE-3122

Mobile Application Development Project

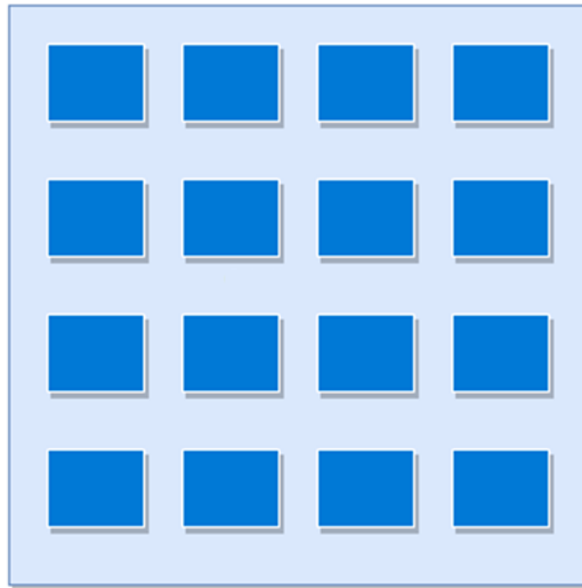
Class-05

Date: 24-12-2020

Android UI Layouts

Android GridView Layout:

- **GridView** is a **ViewGroup** that is used to display items in a **two-dimensional, scrollable grid** and grid items are automatically inserted to the **gridview** layout using a list adapter.
- The adapter pulls data from sources such as an array or database and converts each item into a result view and that's placed into the list.



GridView Attributes:

Sr.No	Attribute & Description
1	android:id This is the ID which uniquely identifies the layout.
2	android:columnWidth This specifies the fixed width for each column. This could be in px, dp, sp, in, or mm.
3	android:gravity Specifies the gravity within each cell. Possible values are top, bottom, left, right, center, center_vertical, center_horizontal etc.
4	android:horizontalSpacing Defines the default horizontal spacing between columns. This could be in px, dp, sp, in, or mm.
5	android:numColumns Defines how many columns to show. May be an integer value, such as "100" or "auto_fit" which means display as many columns as possible to fill the available space.
6	android:stretchMode Defines how columns should stretch to fill the available empty space, if any. This must be either of the values – • none – Stretching is disabled. • spacingWidth – The spacing between each column is stretched. • columnWidth – Each column is stretched equally. • spacingWidthUniform – The spacing between each column is uniformly stretched.
7	android:verticalSpacing Defines the default vertical spacing between rows. This could be in px, dp, sp, in, or mm.

Android GridView Example:

Following is the simple example showing user details using **GridView** and showing the position of a particular image when clicking on it in android applications.

Step-1: Create a new android application using android studio and give names as **GridView**.

Step-2: Add some sample images to project **/res/drawable** directory to show the images in **GridView**.

Step-3: Now open an **activity_main.xml** file from **/res/layout** path and write the code like as shown below

```
<?xml version="1.0" encoding="utf-8"?>
<GridView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/gridview"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:columnWidth="110dp"
    android:numColumns="auto_fit"
    android:verticalSpacing="10dp"
    android:horizontalSpacing="10dp"
    android:stretchMode="columnWidth"
    android:gravity="center" />
```

Step-4: Once we are done with creation of layout, we need to create a custom adapter (**ImageAdapter.java**) by extending it using **BaseExtender** to show all the items in the grid, for that right click on **/java/com.example.gridview** folder then go to “New” then “Java Class”, Give name as **ImageAdapter.java** and click **OK**.

```
package com.example.gridview;
import android.content.Context;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.GridView;
import android.widget.ImageView;

public class ImageAdapter extends BaseAdapter {
    private Context mContext;
    public ImageAdapter(Context c) {
        mContext = c;
    }
```

```

    }
    public int getCount() {
        return thumbImages.length;
    }
    public Object getItem(int position) {
        return null;
    }
    public long getItemId(int position) {
        return 0;
    }
    // create a new ImageView for each item referenced by the Adapter
    public View getView(int position, View convertView, ViewGroup parent) {
        ImageView imageView = new ImageView(mContext);
        imageView.setLayoutParams(new GridView.LayoutParams(200, 200));
        imageView.setScaleType(ImageView.ScaleType.CENTER_CROP);
        imageView.setPadding(8, 8, 8, 8);
        imageView.setImageResource(thumbImages[position]);
        return imageView;
    }
    // Add all our images to arraylist
    public Integer[] thumbImages = {
        R.drawable.img1, R.drawable.img2,
        R.drawable.img3, R.drawable.img4,
        R.drawable.img5, R.drawable.img6,
        R.drawable.img7, R.drawable.img8,
        R.drawable.img1, R.drawable.img2,
        R.drawable.img3, R.drawable.img4,
        R.drawable.img5, R.drawable.img6,
        R.drawable.img7, R.drawable.img8,
        R.drawable.img1, R.drawable.img2,
        R.drawable.img3, R.drawable.img4,
        R.drawable.img5
    };
}

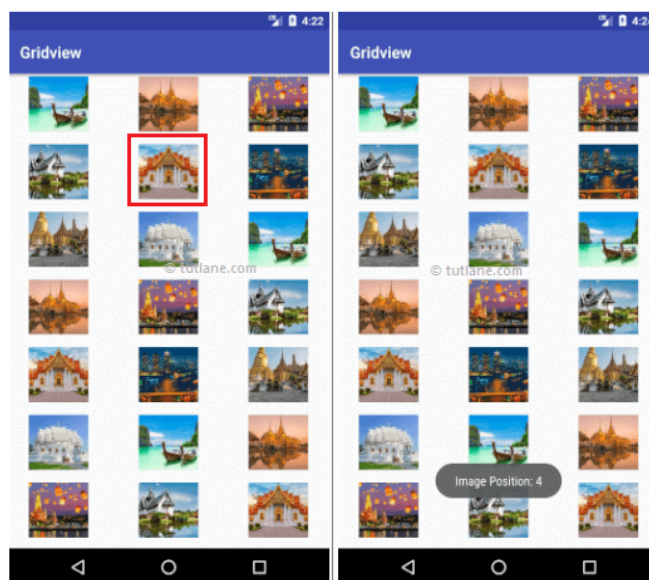
```

Step-5: Now we will bind images to our **GridView** using our custom adapter (**ImageAdapter.java**), in main activity file **MainActivity.java**

```
package com.example.gridview
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        GridView gv = (GridView) findViewById(R.id.gvDetails);
        gv.setAdapter(new ImageAdapter(this));
        gv.setOnItemClickListener(new AdapterView.OnItemClickListener() {
            public void onItemClick(AdapterView<?> parent, View
v, int position, long id) {
                Toast.makeText(MainActivity.this, "Image Position: " +
position, Toast.LENGTH_SHORT).show();
            }
        });
    }
}
```

Output of Android GridView Example:



Android WebView Layout:

- **WebView** is an extension of **View** class and it is used to show the static HTML web pages content or remote web pages content with URL in android applications.
- Generally, in android, **WebView** will act as an embedded browser to include the web pages content in our activity layout and it won't contain any features of normal browsers, such as address bar, navigation controls, etc.
- **WebView** is useful to include and show the content of other web pages or application content in our required pages, such as an end-user agreement, etc.



1. Showing a static HTML content in WebView:

Step-1: Now open an **activity_main.xml** file from **/res/layout** path and add a **Webview** in our xml layout.

```
<?xml version="1.0" encoding="utf-8"?>
<WebView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/webview"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent" />
```

Step-2: Once we are done with adding a **WebView** to the layout, now we will show the static HTML content in **WebView**, for that open main activity file **MainActivity.java**

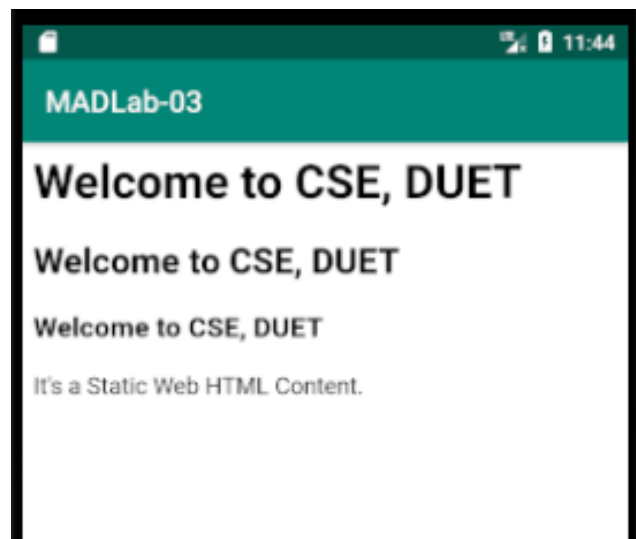
```
package com.example.webview;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.webkit.WebView;

public class WebViewActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_web_view);
        WebView wv = (WebView) findViewById(R.id.webview);
        String customHtml = "<html><body><h1>Welcome to CSE, DUET</h1>" +
            "<h2>Welcome to CSE, DUET</h2><h3>Welcome to CSE,"
DUET</h3>" +
            "<p>It's a Static Web HTML Content.</p>" +
            "</body></html>";
        wv.loadData(customHtml, "text/html", "UTF-8");
    }
}
```

Step-3: When we run above example using an android virtual device (AVD) we will get a result like as shown below.



2. Load Remote URL on WebView:

Step-2: By using **LoadURL** property of **WebView** we can load remote URL content in our android applications. To show the remote URL content in webview modify **MainActivity.java** file code as shown below.

```
package com.example.webview;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.webkit.WebView;

public class WebViewActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_web_view);
        WebView webView = (WebView) findViewById(R.id.webview);
        webView.getSettings().setJavaScriptEnabled(true);
        webView.loadUrl("https://www.duet.ac.bd");
    }
}
```

Step-3: To load the remote web URL content, our application must have an access to the internet. We need to set internet access permissions like as shown below.

```
<!--Add this before application tag in AndroidManifest.xml-->
<uses-permission android:name="android.permission.INTERNET" />
```



Step-4: When we run above example using android virtual device (AVD) we will get a result like as shown below.



Android Intent

Intent:

- **Intent** is a **messaging object**, is used to request **an action from another app components** such as activities, services, broadcast receivers, and content providers.
- **Intents** will help us to maintain the **communication between app components** from the **same application** as well as with the components of **other applications**.
- **Intents** are the objects of ***android.content.Intent*** types and intents are mainly useful **to perform the following operations**.

Component	Description
Starting an Activity	By sending an Intent object to startActivity() method we can start a new Activity or existing Activity to perform required things.
Starting a Service	By sending an Intent object to startService() method we can start a new Service or send required instructions to an existing Service.
Delivering a Broadcast	By sending an Intent object to sendBroadcast() method we can deliver our messages to other app broadcast receivers.

Parts of an Intent Object:

- **Intent object** contains the information about **which component/activity** to start and **the action** to be executed by the recipient component.
- **Following characteristics** helps the android system to understand which component should start.

1. Component Name:

- It defines **the name of the component** to start.
- By using the component name android system will **deliver intent to the specific app component defined by the component name**.

- We can specify the component name for intent by using a **fully qualified class name** of the target component and package name, for example, *com.example.sampleActivity*.
- We can set the component name by using *setComponent()*, *setClass()*, *setClassName()* or by using the **Intent constructor**.

```
public void sendMessage(View view)
{
    Intent intent = new Intent( packageContext this, DisplayMessageActivity.class);
    EditText editText = (EditText) findViewById(R.id.editText);
    String message = editText.getText().toString();
    intent.putExtra(EXTRA_MESSAGE, message);
    startActivity(intent);
}
```

2. Action:

- It defines the **name of the action** to be performed to start an activity.
- We can specify the action name of intent by using *setAction()* or with an **Intent constructor**.
- The following are some of the common actions to start an activity.

Action	Description
ACTION_VIEW	We can use this action in intent with startActivity(), when we have information that activity can show to the user.
ACTION_SEND	We can use this action in intent with startActivity(), when we have some data that the user can share through another app such as an email app, social sharing app.

3. Data:

- It specifies a **type of data** to an intent filter.
- When we create an intent, it's important to specify the type of data (MIME type) in addition to its URI.
- By specifying a MIME type of data, it helps the android system to decide which is the best component to receive our intent.

```
Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("http://www.duet.ac.bd"));
startActivity(intent);
```

4. Category:

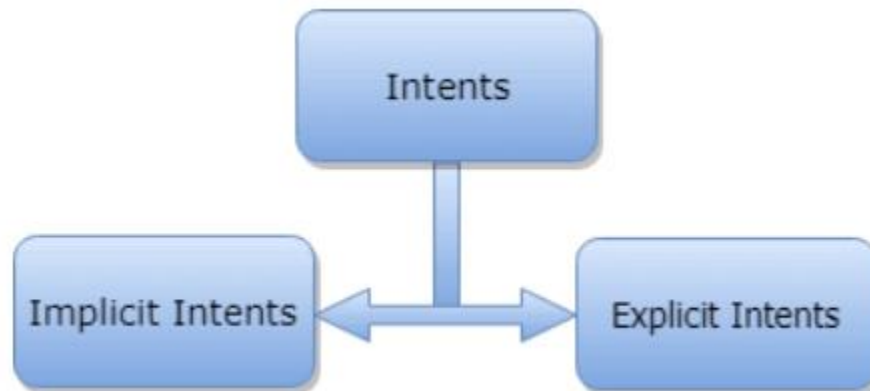
- **Category** is optional for intents and it specifies the additional information about **the type of component** that should handle an intent.
- We can specify a category for intent by using ***addCategory()***.
- By specifying a MIME type of data, it helps the android system to decide which is the best component to receive our intent.

```
<activity android:name=".GridViewActivity" >
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

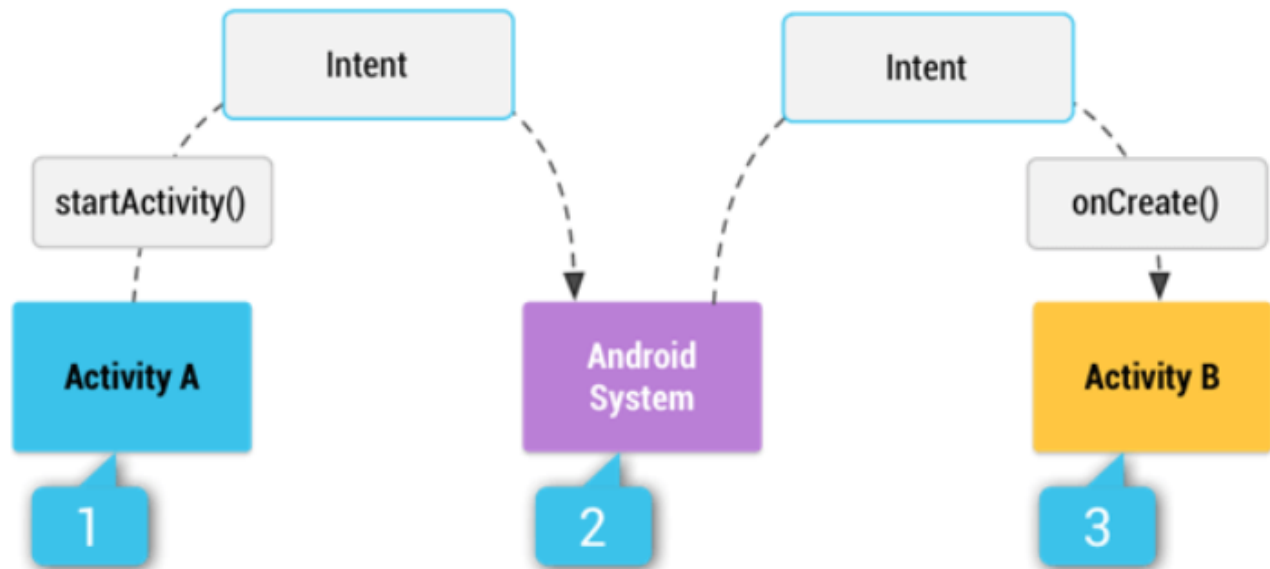
Android Intent Types:

There are two types of intents available in android, those are **Implicit Intents** and **Explicit Intents**.



Android Implicit Intents:

- **Implicit Intents** won't specify any name of the component to start instead, it **declares an action** to perform and it allows a component **from other apps** to handle it.
- For example, by using implicit intents we can **request another app to show the location** details of the user or etc.



If you observe the above image **Activity A** creates an **intent** with the required action and sends it to an android system using the **startActivity()** method. The android system will search for **an intent filter which defined in the manifest file of other apps** that matches the intent in all apps. Whenever the match found **the system starts matching activity (Activity B)** by invoking the **onCreate()** method.

Following is the simple **code snippet of implicit intent** in the android application.

```
Intent intent=new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("http://www.duet.ac.bd"));
startActivity(intent);
```

Implicit Intent Example:

1. Create a new android application using android studio and open an **activity_main.xml** file from **\src\main\res\layout** path.

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="com.tutlane.intents.MainActivity">
    <EditText
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/urlText"
        android:layout_alignParentTop="true"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="100dp"
        android:ems="10" />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/btnNavigate"
        android:layout_below="@+id/urlText"
        android:text="Navigate"
        android:onClick="navigateURL"
        android:layout_centerHorizontal="true" />
</RelativeLayout>
```

2. Now open the main activity file **MainActivity.java** from **\src\main\java\com\tutlane\com.example.myApplication** path and write the following code to open a new browser window on button click to load given url.

```
import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;

public class ImplicitActivity extends AppCompatActivity {
```

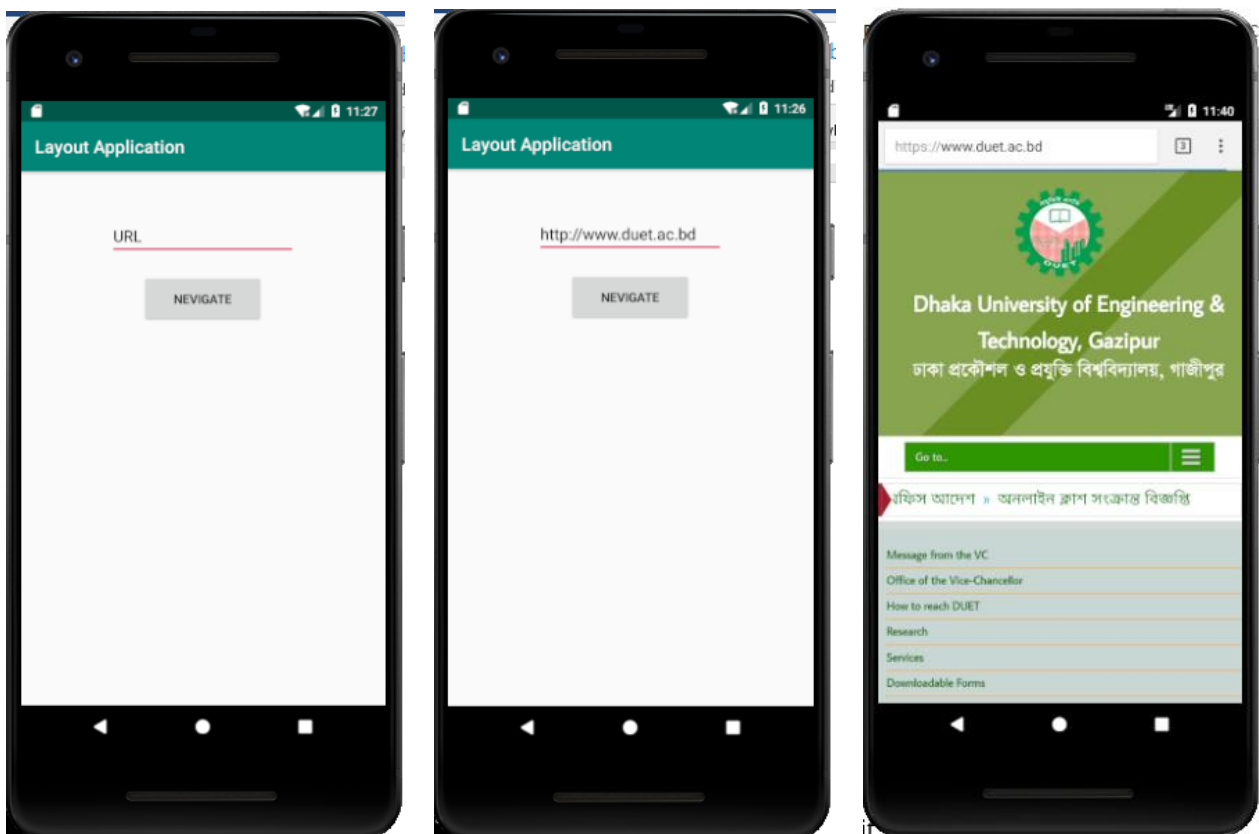
```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_implicit);
}

public void navigateURL(View view)
{
    EditText editText = (EditText) findViewById(R.id.editText5);
    String url = editText.getText().toString();
    Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse(url));
    startActivity(intent);
}
}

```

3. When we run the above example using the android virtual device (AVD) we will get a result like as shown below.



When we enter the website URL and click on the button it will open a website in a new browser window in the same application.

Android Explicit Intents:

- **Explicit intents** explicitly specify the name of the component to be invoked by activity.
- We use **explicit intents** to start a component in **our own app**.
- By using explicit intents, **we can send or share data/content from one activity to another activity** based on our requirements.
- For example, we can start a new activity in response to a user action using explicit intents.
- To create an **Explicit Intent**, we need to define **the component name** for an Intent object.

Following is the simple **code snippet of explicit intent** in the android application.

```
Intent intent = new Intent(this, ActivityView.class);  
intent.setData(Uri.parse("http://www.duet.ac.bd"));  
startService(intent);
```

If you observe above explicit intent, we have specified an app context and **external class name (ActivityView)** object. The intent explicitly starts the **ActivityView** class in the app.

Explicit Intent Example:

1. Create a new android application using android studio and open an **activity_main.xml** file from **\src\main\res\layout** path. Add following view elements.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="24dp"
        android:layout_marginLeft="24dp"
        android:layout_marginTop="50dp"
        android:layout_marginEnd="32dp"
        android:layout_marginRight="32dp"
        android:onClick="sendMessage"
        android:text="Button"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@+id/editText"
        app:layout_constraintTop_toTopOf="parent" />

    <EditText
        android:id="@+id/editText"
        android:layout_width="217dp"
        android:layout_height="49dp"
        android:layout_marginStart="32dp"
        android:layout_marginLeft="32dp"
        android:ems="10"
        android:inputType="textPersonName"
        android:text="Name"
        app:layout_constraintBottom_toBottomOf="@+id/button"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="@+id/button"
        app:layout_constraintVertical_bias="0.0" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

2. Now open our main activity file **MainActivity.java** from **\src\main\java\com.example.myapplication** path and write the code like as shown below

```
package com.example.myapplication;

import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;

public class MainActivity extends AppCompatActivity {
    public static final String EXTRA_MESSAGE =
"com.example.myapplication.MESSAGE" ;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    public void sendMessage(View view)
    {
        Intent intent = new Intent(this, DisplayMessageActivity.class);
        EditText editText = (EditText) findViewById(R.id.editText);
        String message = editText.getText().toString();
        intent.putExtra(EXTRA_MESSAGE, message);
        startActivity(intent);
    }
}
```

3. Now add second activity (**DisplayMessageActivity**) to receive the data send by the MainActivity. Add following code in **activity_display_message.xml** layout file from **\src\main\res\layout** path.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".DisplayMessageActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="129dp"
        android:layout_height="38dp"
        android:layout_marginTop="50dp"
        android:text="TextView"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.5"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

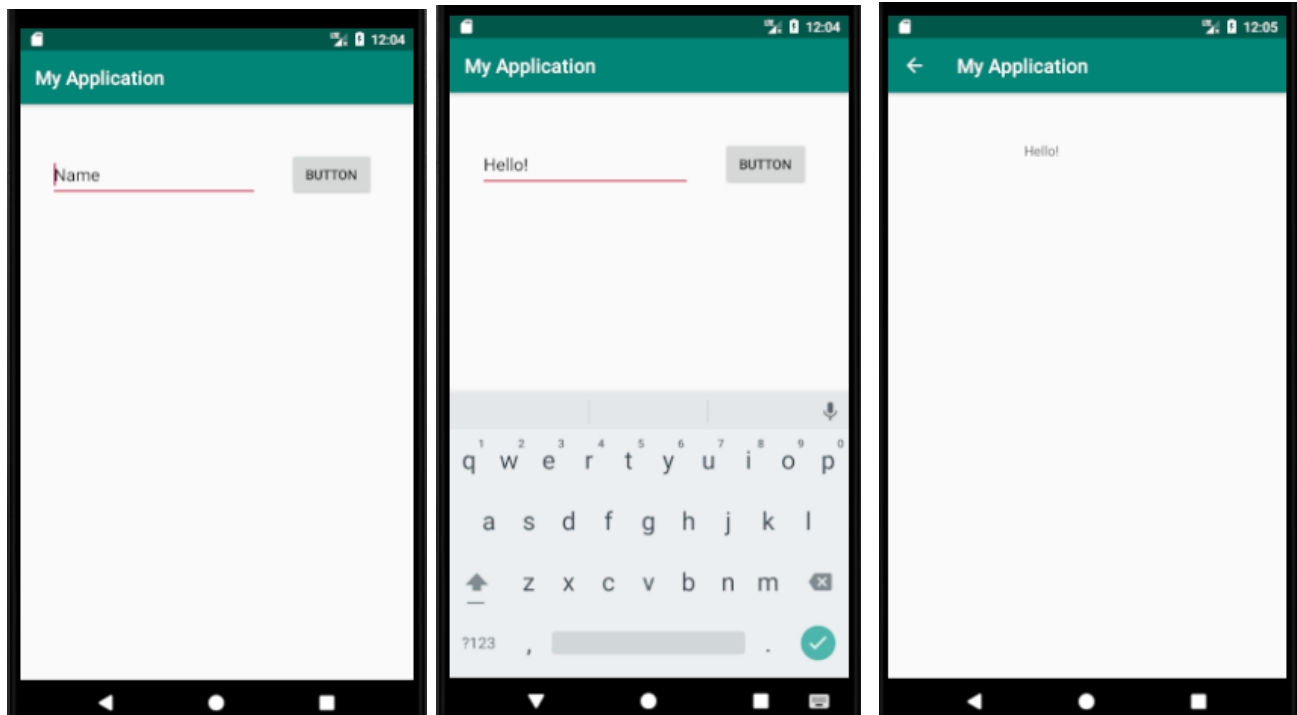
4. Write the code like as shown below in **DisplayMessageActivity.java** file in the **\src\main\java\com.example.myapplication** path to receive the data sent by MainActivity.java.

```
import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;
import android.os.Bundle;
import android.widget.TextView;

public class DisplayMessageActivity extends AppCompatActivity {

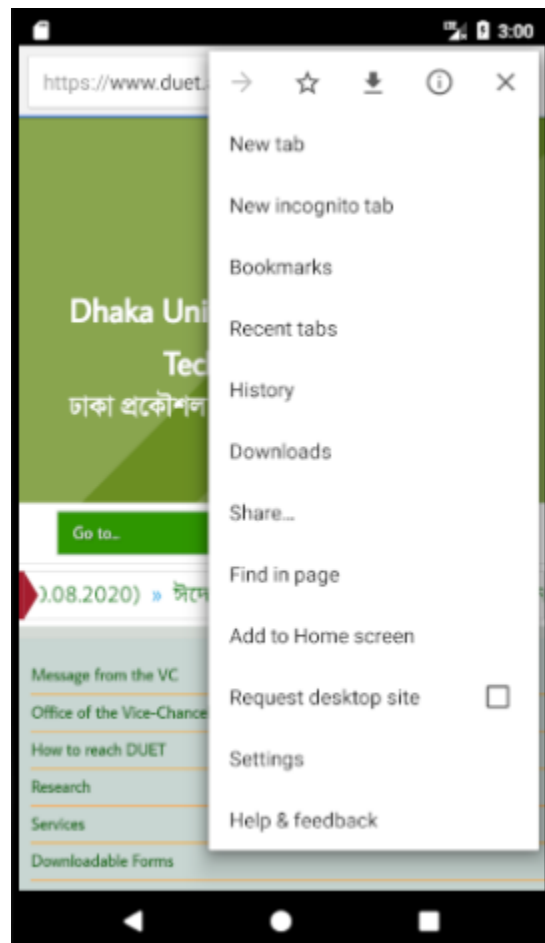
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_display_message);
        Intent intent = getIntent();
        String message = intent.getStringExtra(MainActivity.EXTRA_MESSAGE);
        TextView textView = findViewById(R.id.textView);
        textView.setText(message);
    }
}
```

5. When we run above example in the android emulator, we will get a result like as shown below.



Android Menus:

- **Menu** is a part of the user interface (UI) component which is used to handle some common functionality around the application.
- By using Menus in our applications, we can provide better and consistent user experience throughout the application.
- In android, we can define a Menu **in separate XML file** and use that file in our activities or fragments based on our requirements.
- Following is the pictorial representation of using menus in the android application.



Define Menu in XML File:

- Android provides a standard XML format to define menu items.
- Instead of building a menu in our activity's code, we should define a menu and all its items in an XML menu resource and load menu resource as a Menu object in our activity.
- To define menu, we need to create a new folder **menu** inside of our project resource directory (**res/menu/**) and add a new XML file to build the menu with the following elements.

Element	Description
<menu>	It's a root element to define a Menu in XML file and it will hold one or more and elements.
<item>	It is used to create a menu item and it represents a single item on the menu. This element may contain a nested <menu> element in order to create a submenu.
<group>	It's an optional and invisible for <item> elements. It is used to categorize the menu items so they share properties such as active state and visibility.

Following is the example of defining a menu in an XML file (**menu_example.xml**).

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/mail"
        android:icon="@drawable/ic_mail"
        android:title="@string/mail" />
    <item android:id="@+id/upload"
        android:icon="@drawable/ic_upload"
        android:title="@string/upload"
        android:showAsAction="ifRoom" />
    <item android:id="@+id/share"
        android:icon="@drawable/ic_share"
        android:title="@string/share" />
</menu>
```

The **<item>** element in **menu** supports different type of attributes to define item's behaviour and appearance. Following are some commonly used **<item>** attributes in android applications.

Attribute	Description
android: id	It is used to uniquely identify an element in the application.
android:icon	It is used to set the item's icon from drawable folder.
android: title	It is used to set the item's title
android:showAsAction	It is used to specify how the item should appear as an action item in the app bar.

In case if we want to add **submenu** in **menu** item, then we need to add a **<menu>** element as the child of an **<item>**. Following is the example of defining a submenu in menu item.

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/file"
        android:title="@string/file" >
        <!-- "file" submenu -->
        <menu>
            <item android:id="@+id/create_new"
                android:title="@string/create_new" />
            <item android:id="@+id/open"
                android:title="@string/open" />
        </menu>
    </item>
</menu>
```

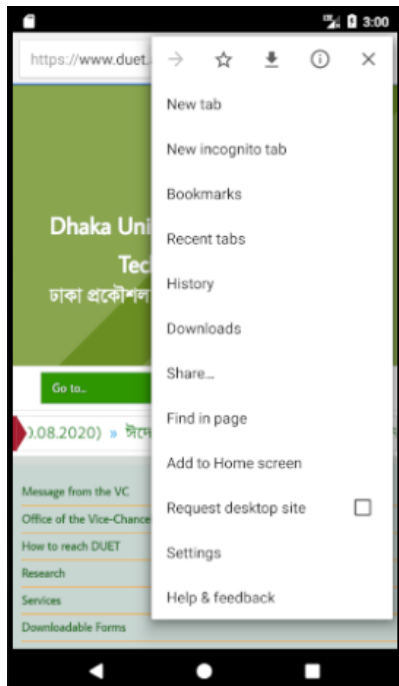
Load Menu from an Activity:

Once we are done with creation of menu, we need to load the menu resource from our activity using **MenuInflater.inflate()** like as shown below.

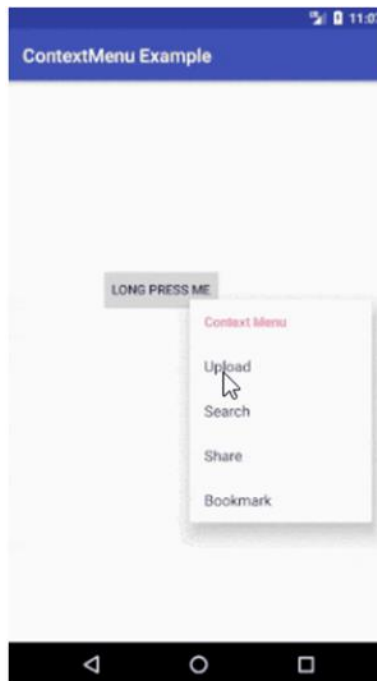
```
@Override
public void onCreateContextMenu(ContextMenu menu) {
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.menu_example, menu);
}
```

Different Types of Menus:

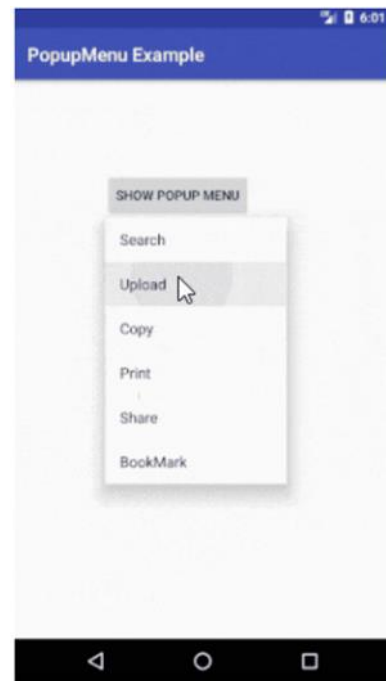
- In android, we have a three-fundamental type of Menus available to define a set of options and actions in our android applications.
- The following are the commonly used Menus in android applications.
 - Options Menu
 - Context Menu
 - Popup Menu



Options Menu



Context Menu

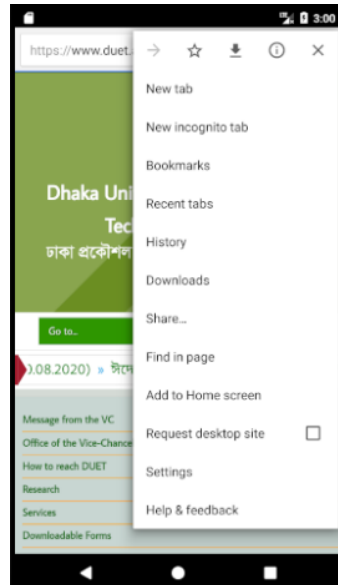


Popup Menu

Options Menu with Examples:

- In android, **Options Menu** is a primary collection of menu items for an activity and it is useful to implement actions that have a global impact on the app, such as Settings, Search, etc.

Following is the pictorial representation of using **Options Menu** in our android applications.



Create Android Options Menu in XML File:

In android, to define **options menu**, we need to create a new folder **menu** inside of our project resource directory (**res/menu/**) and add a new XML (**menu_example**) file to build the menu.

Following is the example of defining a menu in an XML file (**menu_example.xml**).

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/mail"
        android:icon="@drawable/ic_mail"
        android:title="@string/mail" />
    <item android:id="@+id/upload"
        android:icon="@drawable/ic_upload"
        android:title="@string/upload"
        android:showAsAction="ifRoom" />
    <item android:id="@+id/share"
        android:icon="@drawable/ic_share"
        android:title="@string/share" />
</menu>
```

Load Options Menu from an Activity:

To specify the options menu for an activity, we need to override ***onCreateOptionsMenu()*** method and load the defined menu resource using ***MenuInflater.inflate()*** like as shown below.

```
@Override
public void onCreateOptionsMenu(ContextMenu menu) {
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.menu_example, menu);
}
```

Handle Options Menu Click Events:

In android, we can handle options menu item click events using the ***onOptionsItemSelected()*** event method.

Following is the example of handling a options menu item click event using ***onOptionsItemSelected()***.

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    switch (item.getItemId()) {
        case R.id.mail:
            // do something
            return true;
        case R.id.share:
            // do something
            return true;
        default:
            return super.onOptionsItemSelected(item);
    }
}
```

Android Options Menu Example:

1. Create XML menu file:

- In android, to define **options menu**, we need to create a new folder **menu** inside of our project resource directory (**res/menu/**) and add a new XML (**options_menu.xml**) file to build the menu.
- Now open newly created xml (**options_menu.xml**) file and write the code like as shown below.

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/mail"
        android:title="Mail" />
    <item android:id="@+id/upload"
        android:title="Upload" />
    <item android:id="@+id/share"
        android:title="Share" />
    <item android:id="@+id/details"
        android:title="Details" />
</menu>
```

2. Load menu in Activity:

- Once we are done with creation of menu, we need to load this menu XML resource from our activity using **onCreateOptionsMenu()** callback method.
- Open main activity file **MainActivity.java** from **\java\com.tutlane.optionsmenu** path and write the code like as shown below.

```
import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;
import android.os.Bundle;
import android.view.ContextMenu;
import android.view.Menu;
import android.view.MenuInflater;
import android.view.MenuItem;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

```

@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.menu_exm, menu);
    return true;
}

@Override
public boolean onOptionsItemSelected(MenuItem item) {
    Toast.makeText(this, "Selected Item: " +item.getTitle(),
Toast.LENGTH_SHORT).show();
    switch (item.getItemId()) {
        case R.id.mail:
            // do your code
            return true;
        case R.id.upload:
            // do your code
            return true;
        case R.id.share:
            // do your code
            return true;
        case R.id.details:
            // do your code
            Intent intent = new Intent(MainActivity.this,
DetailsActivity.class);
            startActivity(intent);
            return true;
        default:
            return super.onOptionsItemSelected(item);
    }
}
}

```

Output of Options Menu Example:

When we run above example using an android virtual device (AVD) we will get a result like as shown below. If you click Details menu the application will shoe details page.

